

CSCE 632 ACCESSIBLE COMPUTING · MODULE 09 · LECTURE NOTES

Testing

What each method catches, measured.

MODULE OUTCOMES

- State what automated testing can and cannot detect.
- Describe a manual testing procedure.
- Give examples of conformant but unusable interfaces.
- Explain testing with disabled participants, and pay for it.
- Build a testing plan with stated coverage.

1 Automated coverage

AUTOMATED TOOLS CAN DETECT ROUGHLY A THIRD OF THE SUCCESS CRITERIA, AND THAT IS THE OPTIMISTIC FRAMING

Published estimates put the fully automatable criteria at around 30% or less, depending on how generously you count partial detection — **and the automatable ones are precisely the mechanical ones: missing alt attributes, computed contrast ratios, missing form labels, invalid ARIA attributes, duplicate ids.**

Which are real defects and are well worth catching cheaply — a checker in continuous integration prevents regressions on everything it can see, and that is a genuine and worthwhile use that this module is not arguing against.

But a tool cannot judge whether the alt text is *correct*, whether a heading structure is *meaningful*, whether a focus order is *sensible*, whether a live region fires at the *right* time, or whether a task is *completable at all* — and those are where the real failures are (§3).

So 'zero errors' means 'no detectable errors in about a third of the criteria' — which is how false conformance claims happen without anybody intending to lie (Module 07 §2): a green dashboard is read as a conformance result, and nothing in the dashboard says otherwise.

Method	What it finds	What it costs
Automated checker	Mechanical defects; roughly a third of the criteria, partially.	Seconds. Put it in continuous integration.
Keyboard-only pass	Operability, focus visibility, focus order, keyboard traps.	Minutes, and the highest yield per minute in the course.

Method	What it finds	What it costs
Screen reader pass	Semantics, naming, announced change, structure.	Longer, and it needs some skill with the tool.
Zoom and reflow pass	Layout failures at 200% and 400%, distant dependencies.	Seconds.
Expert review	Most of the remaining criteria, and the judgement calls.	Hours, and it needs genuine expertise.
Testing with disabled users	The class nothing else finds (§4).	Paid, scheduled, and worth it.

The methods are complementary and not substitutable — each row finds a class of defect the rows above it structurally cannot, which is why a testing plan names all six rather than choosing among them. The complementary-not-substitutable point is the module's organising claim, and it is the answer to 'we run axe in CI'.

2 The manual pass

- 1 RUN THE CHECKER FIRST
it is free, and it clears the mechanical noise so that your attention is spent on the things only you can assess
- 2 KEYBOARD ONLY
the five properties (Module 05 section 2)
on the three most important tasks
- 3 ZOOM TO 400% AND CHECK REFLOW
plus the magnifier, for distant dependencies
- 4 CONTRAST AND GREYSCALE
numeric, and fast
- 5 SCREEN READER
the same three tasks, eyes closed
recording what is announced at each step
- 6 READ THE TREE
role, name, value, state (Module 08)
for every control in those tasks
- 7 THEN THE CONTENT
headings, link text, language, error
messages, and alt text quality

Checker first, then keyboard, then screen reader — the cheap and broad passes before the slow and deep ones, so that the expensive attention is not spent on defects a tool would have found. Recording what is announced at each step of the screen reader pass is the part to be disciplined about: the transcript is the

evidence, and the gap between it and what you assumed is the finding (Project 1's requirement).

3 Conformant and unusable

- **A form where every field is labelled, the labels are programmatically associated, and the labels read 'Field 1' through 'Field 14' — fully conformant, and completely unusable, and nothing in the standard says otherwise.**
- **Alt text reading 'image' on forty images — present, accurate as a description of what it is, and entirely useless — and no tool can tell the difference between that and a good description, because both are non-empty strings.**
- **A heading structure that is perfectly nested and describes the layout rather than the content — 'Header', 'Sidebar', 'Content', 'Footer' — which passes every structural check and provides no table of contents at all (Module 03 §2).**
- **A focus order that is complete, visible, and runs right-to-left down the page, visiting the footer between two fields of the same form — every element reachable, and the order meaningless.**
- **And a flow of forty keyboard-accessible steps, each one individually conformant, that is simply not usable by switch access (Module 05 §1's step cost). Each of these passes every automated and most manual criteria — which is what 'necessary, checkable, insufficient' means in practice (Module 07 §3).**

4 Testing with people

TEST WITH DISABLED PARTICIPANTS, PAY THEM PROPERLY, AND RECRUIT FOR THE TECHNOLOGY THEY ACTUALLY USE

Because expertise with the assistive technology is a real skill — a daily screen reader user navigates at speech rates you cannot follow, uses operations you do not know, and finds problems your slow careful pass structurally cannot, as well as routing around ones you would call blockers.

And they are testing your interface, not their ability — which is CSCE 671 Module 09's framing, and it matters more here because the opposite framing is so readily available and so damaging: a participant who struggles has found a defect, not demonstrated a limitation.

Pay market rate for the time, as you would for any specialist consultancy, because that is what it is — unpaid accessibility testing by disabled people is extremely common and is not defensible, and 'we asked the community' is not a testing plan.

And recruit for the technology and the configuration rather than for the diagnosis — 'a daily NVDA user on Windows', 'a switch access user', 'a magnifier user at 400%' are the recruitment criteria (Module 01 §3's capability framing) — and five participants finds most of it, for CSCE 671 Module 09 §3's reasons.

AND THE HONEST TESTING PLAN

States each method, what it covers, and what it does not — which converts a plan into a coverage claim and makes the remaining gap visible rather than implicit.

Puts the automated check in continuous integration, because its real value is preventing regression rather than finding defects — and that value is high, since accessibility defects are reintroduced constantly by people who were not thinking about them.

Schedules the manual passes at a stated cadence, since **a manual audit is a snapshot and interfaces change** — which is Module 12 §4's process argument.

And includes disabled participants, paid — without which the plan has a named gap, which is better than an unnamed one and is exactly what Module 13 §2 asks you to write down: the thing you did not test, stated.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
The WAI evaluation resources and WCAG-EM (free)	§2 — a defined evaluation methodology, including sampling for large sites.
The WebAIM Million (free)	§1 at scale — what automated testing finds across a million home pages, yearly, and the detectable-defect density is the headline.
Easy Checks: a first review of web accessibility (free)	§2's short version — the checks worth running before anything else, and a good basis for Project 1.
Deque's and WebAIM's coverage analyses (free)	§1's number — and read it critically, since the vendors have an interest in the figure being high.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **State the automated coverage figure** and what 'zero errors' therefore means.
- 2 **Run three different checkers** on one page and compare their findings.
- 3 **Count the defects you find manually** that none of them reported.
- 4 **Run §2's seven steps in order** on your own project.
- 5 **Record a screen reader transcript** for one task, step by step.

- 6 **Build a conformant unusable form**, deliberately, and run a checker on it.
- 7 **Find a published site with 'image' alt text** at scale.
- 8 **Write a recruitment criterion** by technology rather than by diagnosis.
- 9 **Price five hours of specialist consultancy** in your market, as a payment baseline.
- 10 **Write a testing plan** with each method's coverage and the remaining gap named.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What fraction of criteria is automatable, and which ones?
- 2 What is the real value of a checker in CI?
- 3 Name four things a tool cannot judge.
- 4 Name the six methods and what each finds.
- 5 Why are they not substitutable?
- 6 Give the manual pass in order, and say why the checker goes first.
- 7 Give three conformant-but-unusable examples.
- 8 Why can no tool detect bad alt text?
- 9 Why test with disabled participants, and what is the framing?
- 10 What does an honest testing plan contain?